Python + PySpark — Brain Warm-up

80 coding questions · try first, answers from page "Answers" · Sagar · Jan 2026

Python

1. **Two sum: return indices of two numbers that add up to target.**
   [2,7,11,15], target=9 → [0,1]

2. **Validate brackets: is the string balanced?**
   '{[()]}' → True, '([)]' → False

3. **Palindrome check ignoring case, spaces and punctuation.**
   'A man, a plan, a canal: Panama' → True

4. **Second largest distinct number without sorting.**
   [10, 5, 10, 8] → 8

5. **Return integers that appear more than once, sorted.**
   [4,3,2,7,8,2,3,1] → [2,3]

6. **Find the missing number in 0..n.**
   [3,0,1] → 2

7. **Move all zeros to the end in place, keeping order.**
   [0,1,0,3,12] → [1,3,12,0,0]

8. **Rotate a list to the right by k steps.**
   [1,2,3,4,5], k=2 → [4,5,1,2,3]

9. **Best time to buy and sell stock (one trade).**
   [7,1,5,3,6,4] → 5 (buy 1, sell 6)

10. **Forward-fill None values with the last seen value (default 0), O(1) space.**
    [None, 5, None, None, 3, None] → [0,5,5,5,3,3]

11. **Character with the longest consecutive run (earliest on ties).**
    'aabbbccdddd' → ('d', 4)

12. **Return bigrams from a sentence.**
    'have free hours and love children' → [('have','free'), ('free','hours'), ...]

13. **Top 3 most common words, case-insensitive, ignoring punctuation.**
    Any paragraph of text

Sagar

14. Words that appear in exactly one of two sentences (case-insensitive).

'I like apples', 'I love apples' → ['like', 'love']

15. Flatten a nested dict into dot keys.

Input {'a':1,'b':{'c':2,'d':{'e':3}}} → {'a':1,'b.c':2,'b.d.e':3}

16. First non-repeating character in a string, in O(n).

'swiss' → 'w'

17. Group anagrams.

['eat','tea','tan','ate','nat'] → [['eat','tea','ate'],['tan','nat']]

18. Merge overlapping intervals.

[[1,3],[2,6],[8,10],[9,12]] → [[1,6],[8,12]]

19. Top K frequent elements without sorting everything.

[1,1,1,2,2,3], k=2 → [1,2]

20. Longest substring without repeating characters.

'abcabcbb' → 3 ('abc')

21. Write a retry decorator with exponential backoff.

Retry an API/DB call up to 3 times: wait 1s, 2s, 4s.

22. Count records per key in a 50 GB log file on an 8 GB machine.

Each line: 'timestamp,user_id,event'. Count events per user.

23. Implement an LRU cache with get/put in O(1).

Capacity 2: put(1), put(2), get(1), put(3) → key 2 is evicted.

24. Trap: what does this print?

```
def add(x, lst=[]):
    lst.append(x)
    return lst
print(add(1), add(2))
```

25. Batch an iterator into chunks of n without loading it all.

range(7), n=3 → [0,1,2] [3,4,5] [6]

26. Rebuild a trip from shuffled (origin, destination) tickets.

[('BLR','DEL'), ('MUM','BLR'), ('DEL','GOA')] → MUM → BLR → DEL → GOA

27. Pick one uniformly random element from a stream you can't store.

Stream of unknown length; one pass only.

Sagar

28. All primes up to N efficiently.

N=20 → [2,3,5,7,11,13,17,19]

29. Fibonacci: make the naive recursive version fast.

fib(50) should return instantly

30. Invert a dict where many keys share the same value.

{'a':1, 'b':2, 'c':1} → {1:['a','c'], 2:['b']}

31. Merge two sorted lists into one sorted list.

[1,3,5], [2,4,6] → [1,2,3,4,5,6]

32. Kth largest element.

[3,2,1,5,6,4], k=2 → 5

33. Parse web logs: count 5xx errors per IP.

1.2.3.4 - - [01/Jan/2026:10:00:00] "GET /api HTTP/1.1" 503 512

34. Read a CSV, validate each row, separate good and bad records.

orders.csv: id, amount, order_date — some rows have text in amount or bad dates

35. Deep merge two nested dicts (second one wins on conflicts).

{'a':{'x':1}}, {'a':{'y':2}} → {'a':{'x':1,'y':2}}

36. Write a context manager that logs how long a block takes.

with timer('load'): run_load()

37. Stream a large JSON-lines file and skip malformed lines.

events.jsonl — 20 GB, some broken lines

38. Trap: what does this print?

```
fs = [lambda: i for i in range(3)]
print([f() for f in fs])
```

39. Trap: what does this print?

```
a = [[1, 2], [3]]
b = a.copy()
b[0].append(9)
print(a)
```

40. Pandas: fill missing prices with the median price of the same category.

df(product, category, price) with NaN prices

Sagar

PySpark

41. Second highest salary in each department (ties allowed).
emp(id, name, dept, salary)

42. Keep only the latest record per key (dedup CDC data).
orders(order_id, status, updated_at) with many versions

43. Users who logged in 3 or more consecutive days.
logins(user_id, login_date), possibly duplicate rows per day

44. Running total and 7-day moving average of sales per store.
sales(store_id, sale_date, amount), one row per day

45. Sessionize clickstream: new session after 30 min of inactivity.
clicks(user_id, ts)

46. Explode a nested JSON array and flatten the structs.
order {id, customer{name,city}, items:[{sku,qty}]}

47. SCD Type 2 load with Delta MERGE.
dim_customer(id, city, start_date, end_date, is_current); new snapshot src(id, city)

48. Fix a skewed join with salting.
big(customer_id, ...) where a few customers have most rows; small dim(customer_id, ...)

49. Customers who bought product A but never product B.
purchases(customer_id, product)

50. Count nulls in every column in one pass.
Any DataFrame with 100+ columns

51. Why is this slow, and how do you fix it?
for c in cols: df = df.withColumn(c, F.trim(c)) # 300 columns

52. Pivot monthly sales: one row per product, a column per month.
sales(product, month, amount)

53. Word count with the DataFrame API.
lines(line STRING)

54. Count events per user, including users with zero events.
users(user_id, name), activity(user_id, event_id, ts)

55. Read a file delimited by '~|'.
id~|name~|city

Sagar

56. **Load a file that has corrupt records without failing the job.**
JSON/CSV where some rows are malformed

57. **Combine two files with different schemas.**
file1(id, name), file2(id, name, email)

58. **Top 3 products by revenue in each category.**
sales(category, product, amount)

59. **Employees who earn more than their manager.**
emp(id, name, salary, manager_id)

60. **Employees earning above their department average.**
emp(name, dept, salary)

61. **Month-over-month revenue growth %.**
orders(order_date, amount)

62. **New vs returning customers per month.**
orders(customer_id, order_date)

63. **Fill missing dates per store with 0 sales.**
sales(store_id, d, amount) — some days missing

64. **Forward-fill nulls with the last known value per device.**
readings(device_id, ts, temp) — temp sometimes null

65. **Split full name into first and last name.**
'Rahul Kumar Sharma' → first='Rahul', last='Kumar Sharma'

66. **Comma-separated tags to rows, and back to an ordered list.**
posts(id, tags='spark,python,sql')

67. **Find fully duplicate rows and how many times each appears.**
Any DataFrame

68. **Compare yesterday's and today's snapshot: inserted, deleted, updated rows.**
old(id, ...), new(id, ...) — same schema

69. **Upsert (SCD Type 1) into a Delta table.**
target(id, ...), updates(id, ...)

70. **Incremental load: only pick rows changed since the last run.**
src(id, ..., updated_at), target table already loaded

Sagar

71. **Parse a JSON string column into columns.**

raw(payload='{"id":1,"items":[{"sku":"A","qty":2}]}')

72. **A daily job writes 2,000 tiny files. Fix it.**

df.write.partitionBy('dt').parquet(path)

73. **Join a 2 TB fact table with a 50 MB dimension table efficiently.**

fact(customer_id, ...), dim(customer_id, segment)

74. **Rewrite this Python UDF with native functions.**

def tier(a): return 'high' if a >= 1000 else 'mid' if a >= 100 else 'low'

75. **When you must use Python logic, write a vectorized UDF.**

Clean a text column with pandas string methods

76. **Kafka is at-least-once. Deduplicate events in a stream.**

stream(event_id, event_ts, ...)

77. **Count page views per 10-minute window in a stream, allowing 15 minutes of late data.**

clicks stream(page, ts)

78. **Median salary per department.**

emp(dept, salary)

79. **Customers with no orders in the last 90 days.**

customers(customer_id), orders(customer_id, order_date)

80. **Find missing IDs in a sequence.**

ids 1..N with gaps, e.g. 1,2,4,7

Sagar

Answers

Python

1. Two sum: return indices of two numbers that add up to target.

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i
```

▶ Hash map of value → index, one pass, O(n). Brute force O(n²) is the answer they want you to improve.

2. Validate brackets: is the string balanced?

```
def valid(s):
    pairs, st = {")": "(", "]": "[", "}": "{"}, []
    for ch in s:
        if ch in "([{":
            st.append(ch)
        elif ch in pairs:
            if not st or st.pop() != pairs[ch]:
                return False
    return not st
```

Sagar

▶ Stack. Don't forget the final check — '((' must return False.

3. Palindrome check ignoring case, spaces and punctuation.

```
def is_pal(s):
    t = [c.lower() for c in s if c.isalnum()]
    return t == t[::-1]
```

▶ Follow-up: O(1) space → two pointers moving inward, skipping non-alphanumeric characters.

4. Second largest distinct number without sorting.

```
def second(nums):
    a = b = float("-inf")
    for x in nums:
        if x > a:
            a, b = x, a
        elif a > x > b:
            b = x
    return None if b == float("-inf") else b
```

▶ One pass, O(n). The 'a > x' check skips duplicates of the max.

5. Return integers that appear more than once, sorted.

```
from collections import Counter
def dups(nums):
    return sorted(k for k, v in Counter(nums).items() if v > 1)
```

▶ Counter is the clean answer. Same idea as finding duplicate keys before a load.

6. Find the missing number in 0..n.

```
def missing(nums):
    n = len(nums)
    return n * (n + 1) // 2 - sum(nums)
```

▶ Sum formula, O(n) time, O(1) space. XOR of indices and values also works and avoids overflow in other languages.

7. Move all zeros to the end in place, keeping order.

```
def move_zeros(a):
    pos = 0
    for i in range(len(a)):
        if a[i] != 0:
            a[pos], a[i] = a[i], a[pos]
            pos += 1
```

Sagar

▶ Two pointers, one pass, no extra list.

8. Rotate a list to the right by k steps.

```
def rotate(a, k):
    k %= len(a)
    a[:] = a[-k:] + a[:-k]
```

▶ k %= len handles k > n. a[:] = modifies in place. In-place O(1) space version: reverse all, then reverse both parts.

9. Best time to buy and sell stock (one trade).

```
def max_profit(prices):
    low, best = float("inf"), 0
    for p in prices:
        low = min(low, p)
        best = max(best, p - low)
    return best
```

▶ Track the lowest price so far. O(n). Sell must come after buy — that's why sorting is wrong.

10. Forward-fill None values with the last seen value (default 0), O(1) space.

```
def ffill(a):
    last = 0
    for i, x in enumerate(a):
        if x is None:
            a[i] = last
        else:
            last = x
    return a
```

▶ Use 'is None', not 'if not x' — that would also replace real zeros.

11. Character with the longest consecutive run (earliest on ties).

```
def longest_run(s):
    best_ch, best, cur, prev = None, 0, 0, None
    for ch in s:
        cur = cur + 1 if ch == prev else 1
        prev = ch
        if cur > best:
            best, best_ch = cur, ch
    return best_ch, best
```

▶ Strict > keeps the earliest character on ties.

12. Return bigrams from a sentence.

```
def bigrams(s):
    w = s.split()
    return list(zip(w, w[1:]))
```

▶ zip of the list with itself shifted by one — a common one-liner trick.

13. Top 3 most common words, case-insensitive, ignoring punctuation.

```
import re
from collections import Counter
def top_words(text, n=3):
    return Counter(re.findall(r"[a-z']+", text.lower())).most_common(n)
```

▶ Lowercase first, regex to tokenize. Follow-up: drop stop-words with a set.

14. Words that appear in exactly one of two sentences (case-insensitive).

```
def mismatched(s1, s2):
    a, b = set(s1.lower().split()), set(s2.lower().split())
    return sorted(a ^ b)
```

▶ Symmetric difference ^. Know set ops: & | - ^.

Sagar

15. Flatten a nested dict into dot keys.

```
def flatten(d, parent=""):
    out = {}
    for k, v in d.items():
        key = f"{parent}.{k}" if parent else k
        if isinstance(v, dict):
            out.update(flatten(v, key))
        else:
            out[key] = v
    return out
```

▶ Recursion on dict values. Follow-up: very deep nesting → use an explicit stack to avoid recursion limit.

16. First non-repeating character in a string, in O(n).

```
from collections import Counter
def first_unique(s):
    c = Counter(s)
    return next((ch for ch in s if c[ch] == 1), None)
```

▶ Two passes, still O(n). We re-scan s, so the original order decides the answer.

17. Group anagrams.

```
from collections import defaultdict
def group(words):
    g = defaultdict(list)
    for w in words:
        g["".join(sorted(w))].append(w)
    return list(g.values())
```

▶ Key = sorted letters. Faster key for long words: a 26-length count tuple → O(n·k) instead of O(n·k log k).

18. Merge overlapping intervals.

```
def merge(iv):
    out = []
    for s, e in sorted(iv):
        if out and s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
```

▶ Sort by start, then extend or append. O(n log n). Same idea as merging overlapping time windows in pipelines.

Sagar

19. Top K frequent elements without sorting everything.

```
import heapq
from collections import Counter
def top_k(nums, k):
    c = Counter(nums)
    return heapq.nlargest(k, c, key=c.get)
```

▶ Heap → O(n log k) instead of O(n log n). Mention bucket sort for O(n).

20. Longest substring without repeating characters.

```
def longest(s):
    seen, left, best = {}, 0, 0
    for i, ch in enumerate(s):
        if seen.get(ch, -1) >= left:
            left = seen[ch] + 1
        seen[ch] = i
        best = max(best, i - left + 1)
    return best
```

▶ Sliding window with last-seen index. O(n), one pass.

21. Write a retry decorator with exponential backoff.

```
import time, functools
def retry(n=3, base=1):
    def deco(f):
        @functools.wraps(f)
        def wrap(*a, **kw):
            for i in range(n):
                try:
                    return f(*a, **kw)
                except Exception:
                    if i == n - 1:
                        raise
                    time.sleep(base * 2 ** i)
        return wrap
    return deco
```

▶ Decorator with arguments = 3 levels. In real code catch only transient errors, and add jitter.

Sagar

22. Count records per key in a 50 GB log file on an 8 GB machine.

```
from collections import Counter
def count(path):
    c = Counter()
    with open(path) as f:
        for line in f:
            c[line.split(",", 2)[1]] += 1
    return c
```

▶ Iterate line by line — never f.read(). If even the counts don't fit in memory: hash-partition users to temp files, or just use Spark.

23. Implement an LRU cache with get/put in O(1).

```
from collections import OrderedDict
class LRU:
    def __init__(self, cap):
        self.cap, self.d = cap, OrderedDict()
    def get(self, k):
        if k not in self.d:
            return -1
        self.d.move_to_end(k)
        return self.d[k]
    def put(self, k, v):
        self.d[k] = v
        self.d.move_to_end(k)
        if len(self.d) > self.cap:
            self.d.popitem(last=False)
```

Sagar

▶ OrderedDict = hash map + doubly linked list. Interviewer may ask you to build that list by hand.

24. Trap: what does this print?

```
# [1, 2] [1, 2]
def add(x, lst=None):
    lst = [] if lst is None else lst
    lst.append(x)
    return lst
```

▶ Default args are evaluated once at definition, so the list is shared. Both calls return the same object.

25. Batch an iterator into chunks of n without loading it all.

```
from itertools import islice
def chunks(it, n):
    it = iter(it)
    while batch := list(islice(it, n)):
        yield batch
```

▶ Generator + islice → constant memory. Used for batched API calls or DB inserts.

26. Rebuild a trip from shuffled (origin, destination) tickets.

```
def itinerary(pairs):
    nxt = dict(pairs)
    start = (set(nxt) - set(nxt.values())).pop()
    path = [start]
    while path[-1] in nxt:
        path.append(nxt[path[-1]])
    return path
```

▶ Start = the only origin that is never a destination. O(n).

27. Pick one uniformly random element from a stream you can't store.

```
import random
def reservoir(stream):
    pick = None
    for i, x in enumerate(stream, 1):
        if random.randrange(i) == 0:
            pick = x
    return pick
```

▶ Reservoir sampling: keep the i-th item with probability 1/i. Generalizes to k items.

28. All primes up to N efficiently.

```
def primes(n):
    is_p = [True] * (n + 1)
    is_p[:2] = [False, False]
    for i in range(2, int(n ** 0.5) + 1):
        if is_p[i]:
            is_p[i*i::i] = [False] * len(range(i*i, n + 1, i))
    return [i for i, p in enumerate(is_p) if p]
```

▶ Sieve of Eratosthenes, O(n log log n). Start crossing out at i*i.

Sagar

29. Fibonacci: make the naive recursive version fast.

```
from functools import lru_cache
@lru_cache(maxsize=None)
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)
```

▶ Memoization turns $O(2^n)$ into $O(n)$. Iterative loop with two variables = $O(1)$ space.

30. Invert a dict where many keys share the same value.

```
from collections import defaultdict
def invert(d):
    inv = defaultdict(list)
    for k, v in d.items():
        inv[v].append(k)
    return dict(inv)
```

▶ A plain {v: k} comprehension silently loses keys — that's the trap.

31. Merge two sorted lists into one sorted list.

```
def merge_sorted(a, b):
    i = j = 0
    out = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            out.append(a[i]); i += 1
        else:
            out.append(b[j]); j += 1
    return out + a[i:] + b[j:]
```

▶ Two pointers, $O(n + m)$. heapq.merge does this lazily for k sorted files — external sort idea.

32. Kth largest element.

```
import heapq
def kth_largest(nums, k):
    h = []
    for x in nums:
        heapq.heappush(h, x)
        if len(h) > k:
            heapq.heappop(h)
    return h[0]
```

▶ Min-heap of size k → $O(n \log k)$, works on a stream. Quickselect is $O(n)$ average.

Sagar

33. Parse web logs: count 5xx errors per IP.

```
import re
from collections import Counter
pat = re.compile(r'^(\S+) .*?" (\d{3}) ')
def errors(lines):
    c = Counter()
    for line in lines:
        m = pat.match(line)
        if m and m.group(2).startswith("5"):
            c[m.group(1)] += 1
    return c
```

▶ Compile the regex once outside the loop. Skip lines that don't match instead of crashing.

34. Read a CSV, validate each row, separate good and bad records.

```
import csv
from datetime import date
def validate(path):
    good, bad = [], []
    with open(path, newline="") as f:
        for row in csv.DictReader(f):
            try:
                row["amount"] = float(row["amount"])
                row["order_date"] = date.fromisoformat(row["order_date"])
                good.append(row)
            except (ValueError, TypeError):
                bad.append(row)
    return good, bad
```

▶ Quarantine bad rows, don't fail the whole load. Catch specific exceptions, not bare except.

35. Deep merge two nested dicts (second one wins on conflicts).

```
def deep_merge(a, b):
    out = dict(a)
    for k, v in b.items():
        if isinstance(v, dict) and isinstance(out.get(k), dict):
            out[k] = deep_merge(out[k], v)
        else:
            out[k] = v
    return out
```

▶ Real use: merging default config with environment config. Doesn't mutate the inputs.

36. Write a context manager that logs how long a block takes.

```
import time
from contextlib import contextmanager
@contextmanager
def timer(name):
    t = time.perf_counter()
    try:
        yield
    finally:
        print(f"{name}: {time.perf_counter() - t:.2f}s")
```

▶ try/finally so it logs even on exceptions. perf_counter, not time.time, for durations.

37. Stream a large JSON-lines file and skip malformed lines.

```
import json
def read_jsonl(path):
    with open(path) as f:
        for n, line in enumerate(f, 1):
            try:
                yield json.loads(line)
            except json.JSONDecodeError:
                print(f"bad line {n}")
```

▶ Generator → constant memory. Log the line number so bad records can be traced.

38. Trap: what does this print?

```
# [2, 2, 2]
fs = [lambda i=i: i for i in range(3)]   # [0, 1, 2]
```

▶ Late binding: closures look up i when called, by then the loop ended at 2. Default argument captures the value.

39. Trap: what does this print?

```
# [[1, 2, 9], [3]]
import copy
b = copy.deepcopy(a)
```

▶ copy() is shallow: the inner lists are shared. deepcopy copies nested objects.

40. Pandas: fill missing prices with the median price of the same category.

```
df["price"] = df.groupby("category")["price"] \
                .transform(lambda s: s.fillna(s.median()))
```

▶ transform keeps the original row shape. A global median would be wrong across categories.

Sagar

PySpark

41. Second highest salary in each department (ties allowed).

```
from pyspark.sql import Window as W, functions as F
w = W.partitionBy("dept").orderBy(F.desc("salary"))
df.withColumn("r", F.dense_rank().over(w)) \
  .filter("r = 2").drop("r")
```

▶ dense_rank, not row_number — ties share a rank. Follow-up: Nth highest → filter r = N.

42. Keep only the latest record per key (dedup CDC data).

```
w = W.partitionBy("order_id").orderBy(F.desc("updated_at"))
latest = df.withColumn("rn", F.row_number().over(w)) \
           .filter("rn = 1").drop("rn")
```

▶ row_number here — exactly one row per key. dropDuplicates() can't choose WHICH row survives.

43. Users who logged in 3 or more consecutive days.

```
d = df.select("user_id", "login_date").distinct()
w = W.partitionBy("user_id").orderBy("login_date")
g = d.withColumn("grp",
      F.date_sub("login_date", F.row_number().over(w)))
g.groupBy("user_id", "grp").count() \
 .filter("count >= 3").select("user_id").distinct()
```

▶ Gaps-and-islands: date minus row number is constant within a streak. Distinct first, or duplicates break it.

44. Running total and 7-day moving average of sales per store.

```
w = W.partitionBy("store_id").orderBy("sale_date")
df.withColumn("running",
      F.sum("amount").over(w.rowsBetween(W.unboundedPreceding, 0))) \
  .withColumn("ma7", F.avg("amount").over(w.rowsBetween(-6, 0)))
```

▶ Missing days? rowsBetween counts rows, not days — use rangeBetween on a day number, or fill dates first.

Sagar

45. Sessionize clickstream: new session after 30 min of inactivity.

```
w = W.partitionBy("user_id").orderBy("ts")
s = df.withColumn("gap",
        F.col("ts").cast("long") - F.lag("ts").over(w).cast("long")) \
  .withColumn("new", F.when(F.col("gap").isNull() | (F.col("gap") > 1800), 1).otherwise(0)) \
  .withColumn("session_id", F.sum("new").over(w))
```

▶ lag → flag → running sum of flags = session number. Classic pattern; also asked in SQL.

46. Explode a nested JSON array and flatten the structs.

```
df.select("id", "customer.*",
          F.explode_outer("items").alias("it")) \
  .select("id", "name", "city", "it.sku", "it.qty")
```

▶ explode_outer keeps orders with empty/null items (explode drops them). struct.* flattens one level.

47. SCD Type 2 load with Delta MERGE.

```
from delta.tables import DeltaTable
t = DeltaTable.forName(spark, "dim_customer")
cur = t.toDF().filter("is_current")
chg = src.join(cur, "id").where(src.city != cur.city).select(src["*"])
staged = chg.withColumn("mk", F.lit(None).cast("long")) \
    .unionByName(src.withColumn("mk", F.col("id")))
t.alias("t").merge(staged.alias("s"), "t.id = s.mk AND t.is_current") \
 .whenMatchedUpdate(condition="t.city <> s.city",
     set={"is_current": "false", "end_date": "current_date()"}) \
 .whenNotMatchedInsert(values={"id": "s.id", "city": "s.city", "start_date": "current_date()",
             "end_date": "null", "is_current": "true"}).execute()
```

▶ Trick: changed rows appear twice — once with a merge key (to close the old row), once with NULL key (to insert the new one). Explain the idea; exact syntax is less important.

48. Fix a skewed join with salting.

```
N = 10
big_s = big.withColumn("salt", (F.rand() * N).cast("int"))
dim_s = dim.withColumn("salt", F.explode(F.sequence(F.lit(0), F.lit(N - 1))))
out = big_s.join(dim_s, ["customer_id", "salt"]).drop("salt")
```

▶ Try first: broadcast(dim) if small, or AQE spark.sql.adaptive.skewJoin.enabled. Salt only hot keys in practice.

49. Customers who bought product A but never product B.

```
a = df.filter("product = 'A'").select("customer_id").distinct()
b = df.filter("product = 'B'").select("customer_id")
a.join(b, "customer_id", "left_anti")
```

▶ left_anti = NOT EXISTS. Cheaper and null-safe compared to NOT IN.

50. Count nulls in every column in one pass.

```
df.select([F.sum(F.col(c).isNull().cast("int")).alias(c)
           for c in df.columns])
```

▶ One select = one job. A Python loop of df.filter(...).count() per column = 100 jobs.

51. Why is this slow, and how do you fix it?

```
df = df.select([F.trim(c).alias(c) if c in cols else c
                for c in df.columns])
```

▶ Each withColumn adds a projection; hundreds of them bloat the plan and slow analysis. One select (or withColumns in Spark 3.3+).

52. Pivot monthly sales: one row per product, a column per month.

```
months = ["Jan", "Feb", "Mar"]
df.groupBy("product").pivot("month", months).sum("amount").fillna(0)
```

▶ Passing the value list skips an extra job to find distinct months and fixes column order.

53. Word count with the DataFrame API.

```
from pyspark.sql import functions as F
df.select(F.explode(F.split(F.lower("line"), r"\W+")).alias("w")) \
  .filter("w != ''") \
  .groupBy("w").count().orderBy(F.desc("count"))
```

▶ split + explode = one row per word. RDD version uses reduceByKey, never groupByKey (shuffles everything).

54. Count events per user, including users with zero events.

```
users.join(activity, "user_id", "left") \
     .groupBy("user_id", "name") \
     .agg(F.count("event_id").alias("events"))
```

▶ Left join keeps inactive users. count(column) ignores nulls → 0; count('*') would wrongly give 1.

55. Read a file delimited by '~|'.

```
df = spark.read.option("header", True) \
        .option("sep", "~|").csv(path)
```

▶ Spark 3+ supports multi-character separators. On Spark 2: read as text and split.

56. Load a file that has corrupt records without failing the job.

```
df = spark.read.schema(schema_with_corrupt_col) \
     .option("mode", "PERMISSIVE") \
     .option("columnNameOfCorruptRecord", "_corrupt_record") \
     .json(path).cache()
bad  = df.filter("_corrupt_record IS NOT NULL")
good = df.filter("_corrupt_record IS NULL").drop("_corrupt_record")
```

▶ Modes: PERMISSIVE / DROPMALFORMED / FAILFAST. Cache before filtering on the corrupt column. Databricks: badRecordsPath.

57. Combine two files with different schemas.

```
out = df1.unionByName(df2, allowMissingColumns=True)
```

▶ union() matches by POSITION and silently mixes columns. unionByName matches by name; missing columns become null.

58. Top 3 products by revenue in each category.

```
rev = df.groupBy("category", "product").agg(F.sum("amount").alias("rev"))
w = W.partitionBy("category").orderBy(F.desc("rev"))
rev.withColumn("r", F.dense_rank().over(w)).filter("r <= 3")
```

▶ Aggregate first, then rank. Say whether ties should be included (dense_rank) or not (row_number).

59. Employees who earn more than their manager.

```
e, m = df.alias("e"), df.alias("m")
e.join(m, F.col("e.manager_id") == F.col("m.id")) \
 .filter("e.salary > m.salary").select("e.name")
```

▶ Self join with aliases — otherwise column names are ambiguous.

60. Employees earning above their department average.

```
w = W.partitionBy("dept")
df.withColumn("dept_avg", F.avg("salary").over(w)) \
  .filter("salary > dept_avg")
```

▶ Window aggregate keeps every row — no groupBy + join back needed.

61. Month-over-month revenue growth %.

```
m = df.groupBy(F.trunc("order_date", "month").alias("month")) \
      .agg(F.sum("amount").alias("rev"))
w = W.orderBy("month")
m.withColumn("prev", F.lag("rev").over(w)) \
 .withColumn("growth_pct", F.round((F.col("rev") - F.col("prev")) / F.col("prev") * 100, 2))
```

▶ Window without partitionBy moves all rows to one partition — fine here only because monthly data is tiny.

62. New vs returning customers per month.

```
o = df.withColumn("month", F.trunc("order_date", "month"))
first = o.groupBy("customer_id").agg(F.min("month").alias("first_month"))
o.join(first, "customer_id") \
 .withColumn("type", F.when(F.col("month") == F.col("first_month"),
"new").otherwise("returning")) \
 .groupBy("month", "type").agg(F.countDistinct("customer_id").alias("customers"))
```

▶ First-purchase month per customer is the key idea. Very common in e-commerce rounds.

63. Fill missing dates per store with 0 sales.

```
b = df.groupBy("store_id").agg(F.min("d").alias("s"), F.max("d").alias("e"))
cal = b.select("store_id",
        F.explode(F.sequence("s", "e", F.expr("interval 1 day"))).alias("d"))
cal.join(df, ["store_id", "d"], "left").fillna(0, ["amount"])
```

▶ Build a calendar with sequence + explode, then left join. Needed before moving averages.

64. Forward-fill nulls with the last known value per device.

```
w = W.partitionBy("device_id").orderBy("ts") \
     .rowsBetween(W.unboundedPreceding, 0)
df.withColumn("temp_filled", F.last("temp", ignorenulls=True).over(w))
```

▶ last(..., ignorenulls=True) over a growing window. Backward fill: first(..., True) with rowsBetween(0, unboundedFollowing).

65. Split full name into first and last name.

```
s = F.split("full_name", " ", 2)
df.withColumn("first", s[0]).withColumn("last", s[1])
```

▶ The limit argument (Spark 3+) keeps the rest of the name together.

Sagar

66. Comma-separated tags to rows, and back to an ordered list.

```
rows = df.select("id", F.explode(F.split("tags", ",")).alias("tag"))
back = rows.groupBy("id").agg(F.sort_array(F.collect_list("tag")).alias("tags"))
```

▶ collect_list order is NOT guaranteed after a shuffle — sort it, or collect structs with a timestamp and sort.

67. Find fully duplicate rows and how many times each appears.

```
df.groupBy(df.columns).count().filter("count > 1")
```

▶ To remove: dropDuplicates(). To remove by key: dropDuplicates(['id']) — but it keeps an arbitrary row.

68. Compare yesterday's and today's snapshot: inserted, deleted, updated rows.

```
cols = [c for c in new.columns if c != "id"]
h = lambda d: d.withColumn("h", F.sha2(F.concat_ws("||",
        *[F.coalesce(F.col(c).cast("string"), F.lit("~")) for c in cols]), 256))
j = h(old).alias("o").join(h(new).alias("n"), "id", "full")
j.withColumn("status",
    F.when(F.col("o.h").isNull(), "inserted")
     .when(F.col("n.h").isNull(), "deleted")
     .when(F.col("o.h") != F.col("n.h"), "updated"))
```

▶ Row hash = one comparison instead of N columns. coalesce nulls first — concat_ws silently skips them.

69. Upsert (SCD Type 1) into a Delta table.

```
from delta.tables import DeltaTable
t = DeltaTable.forName(spark, "target")
t.alias("t").merge(upd.alias("s"), "t.id = s.id") \
 .whenMatchedUpdateAll() \
 .whenNotMatchedInsertAll().execute()
```

▶ Dedup the source first — MERGE fails if multiple source rows match one target row.

70. Incremental load: only pick rows changed since the last run.

```
last = spark.table("target").agg(F.max("updated_at")).first()[0]
inc = src if last is None else src.filter(F.col("updated_at") > F.lit(last))
# then MERGE inc into target
```

▶ Handle the first run (None). Safer in prod: store the watermark in a control table, and use a small overlap + MERGE for late rows.

71. Parse a JSON string column into columns.

```
schema = "id INT, items ARRAY<STRUCT<sku: STRING, qty: INT>>"
df.withColumn("j", F.from_json("payload", schema)) \
  .select("j.id", F.explode("j.items").alias("it")) \
  .select("id", "it.sku", "it.qty")
```

▶ Always give an explicit schema — inference scans the data and drifts. schema_of_json helps you draft it.

72. A daily job writes 2,000 tiny files. Fix it.

```
df.repartition("dt").write.partitionBy("dt").parquet(path)
# or fewer files without full shuffle:
df.coalesce(8).write.parquet(path)
```

▶ repartition by the partition column → ~1 file per partition. coalesce avoids a shuffle but can create uneven files. Delta: OPTIMIZE / auto-compaction.

73. Join a 2 TB fact table with a 50 MB dimension table efficiently.

```
out = fact.join(F.broadcast(dim), "customer_id")
```

▶ Broadcast avoids shuffling 2 TB. Default auto threshold is 10 MB (spark.sql.autoBroadcastJoinThreshold). Never broadcast a big table — driver OOM.

74. Rewrite this Python UDF with native functions.

```
df.withColumn("tier",
    F.when(F.col("amount") >= 1000, "high")
     .when(F.col("amount") >= 100, "mid")
     .otherwise("low"))
```

▶ Python UDFs serialize every row to a Python worker and block optimizations. Native when/otherwise runs in the JVM — often several times faster.

75. When you must use Python logic, write a vectorized UDF.

```
import pandas as pd
@F.pandas_udf("string")
def clean(s: pd.Series) -> pd.Series:
    return s.str.strip().str.lower().str.replace(r"\s+", " ", regex=True)
df.withColumn("clean", clean("text"))
```

▶ pandas_udf uses Arrow and processes batches, much faster than row-by-row udf. Still prefer native functions first.

76. Kafka is at-least-once. Deduplicate events in a stream.

```
deduped = stream.withWatermark("event_ts", "10 minutes") \
                .dropDuplicatesWithinWatermark(["event_id"])
```

▶ Watermark bounds the state. Spark 3.5+; on older versions use dropDuplicates(['event_id', 'event_ts']) with a watermark. Sink should also be idempotent (MERGE).

77. Count page views per 10-minute window in a stream, allowing 15 minutes of late data.

```
agg = clicks.withWatermark("ts", "15 minutes") \
            .groupBy(F.window("ts", "10 minutes"), "page").count()
agg.writeStream.outputMode("append").format("delta") \
   .option("checkpointLocation", "/chk/pv").start("/delta/pv")
```

▶ Append mode emits a window only after the watermark passes it. Events later than 15 min are dropped.

78. Median salary per department.

```
df.groupBy("dept").agg(F.percentile_approx("salary", 0.5).alias("median"))
```

▶ Exact median is expensive at scale; percentile_approx is the standard answer. Spark 3.4+ also has F.median.

79. Customers with no orders in the last 90 days.

```
recent = orders.filter(F.col("order_date") >= F.date_sub(F.current_date(), 90)) \
               .select("customer_id")
customers.join(recent, "customer_id", "left_anti")
```

▶ left_anti = 'not in the other table'. Churn-type question, very common.

80. Find missing IDs in a sequence.

```
lo, hi = df.agg(F.min("id"), F.max("id")).first()
spark.range(lo, hi + 1).join(df, "id", "left_anti")
```

▶ spark.range builds the full sequence (column named id). Alternative: lead(id) - id > 1 to find gap ranges.

— Sagar